



Amazon

Data Engineer Interview Q&A (2–5 Years)

Service-Based Company (Real Mix)

1 Question: Explain the Data Pipeline you built end-to-end.

What they asked me

“Walk me through an end-to-end data pipeline you designed for batch or real-time ingestion.”

What I said

I explained a real AWS pipeline:

Ingestion Layer

- Source: OLTP PostgreSQL + Kinesis Firehose
- Batch: AWS DMS → S3
- Real-time: Firehose → S3 (compressed parquet)

Processing Layer

- Used AWS Glue + PySpark jobs for cleaning, joins, transformations
- Partitioned S3 data by `year/month/day`

Storage Layer

- Raw zone → S3 bucket
- Processed zone → S3 parquet
- Curated → Amazon Redshift (for analytics)

Consumption Layer

- Redshift + Athena + QuickSight dashboards
- APIs used Lambda to serve curated data

Tips

- They LOVE hearing “S3 → Glue → Redshift → QuickSight”.
 - Always mention partitioning + metadata catalogs.
- 

2 Question: How did you optimize a slow AWS Glue job?

What they asked me

“Glue jobs timing out or taking too long — how did you tune them?”

What I said

I gave **5 optimizations**:

1. **Pushdown filters** before writing
2. Replace `groupByKey()` with `reduceByKey()`
3. Increase worker type to G1.X or G2.X
4. Enable **Glue Job Bookmark** to avoid reprocessing
5. Use **dynamic frames** → **data frames** for performance

Scenario:

A 40M row dataset reduced from 28 mins to 7 mins after optimization.

Tips

- Always mention “DynamicFrame → DataFrame conversion”.
- Avoid saying “just increased workers” — weak answer.

3 Question: How do you handle schema evolution in a Data Lake on AWS?

What they asked me

“What happens when source schema changes? How do you manage it?”

What I said

Three techniques:

1. Glue Schema Registry

Stores versioned schema for Kafka/Kinesis.

2. Use AWS Glue jobs to detect new columns

- Add missing columns with default values
- Maintain schema mapping

3. Use Lake Formation + Crawler updates



Crawler detects new partitions but may not handle deep schema drift.

Scenario:

A new column `discount_code` appeared.

I updated the Glue ETL job to add it with default "NA" if missing.

Tips

- Mention that schema drift is **real-world**.
- Never say "Crawler handles everything" — wrong.

4 Question: How do you do incremental loads in AWS?

What they asked me

"We don't want to reload everything. How do you do incremental ingestion?"

What I said

Batch Incremental

- Use DMS CDC to bring only changed records
- Store `last_modified_timestamp` in metadata table
- Filter using Glue jobs:
- `df.filter(df.last_update > watermark_value)`

Real-Time

- Kinesis Data Streams + Lambda → S3
- Process small micro-batches every second

Tips

- Mention DMS CDC → big brownie points.
- Don't say "I reload full tables daily" — this is unacceptable for AWS.

5 Question: Difference between S3, Kinesis, and Kafka?

What they asked me

"If you were to design a streaming system, what would you pick and why?"

What I said

S3



- 
- Storage layer
 - Batch processing
 - Not a streaming engine

Kinesis

- AWS-native streaming
- Real-time
- Managed service
- Easy integration with Redshift, Lambda, Firehose

Kafka on MSK

- More control
- Larger scale
- Higher throughput
- Best for multi-region or cross-cloud setups

Tips

- For Amazon projects, always lean towards **Kinesis** unless large-scale constraints exist.

6 Question: What is the best file format for S3-based Data Lake?

What they asked me

“Do you use CSV, Parquet, ORC, or Avro?”

What I said

Best choice → **Parquet**

- Columnar
- Compressed
- Supports predicate pushdown
- Works fastest with Athena + Redshift Spectrum

Delta Lake / Iceberg / Hudi enable:

- ACID
- Schema evolution
- Time travel
- Better upserts

Tips

- Always mention Parquet + Iceberg/Hudi for 2025.
- 

- CSV only for ingestion — never for analytics.

7 Question: What is Amazon Redshift Spectrum? When do you use it?

What they asked me

“When do you use Redshift Spectrum instead of loading data inside Redshift tables?”

What I said

Use **Redshift Spectrum** when:

- Data is large & stored in S3
- You want to query without loading into Redshift
- Ideal for cold/historical data
- Saves storage cost
- Works best with Parquet

Scenario:

We kept last 7 days in Redshift tables

Older data (2 years) in S3

Queried through Spectrum → cut storage costs by 40%.

Tips

- Always highlight cost optimization — Amazon loves that answer.

8 Question: How do you secure data in S3?

What they asked me

“How do you ensure S3 data is secure?”

What I said

I mentioned:

- ✓ **Bucket Policies with least privilege**
 - ✓ **IAM Roles (not access keys)**
 - ✓ **Encryption (SSE-S3, SSE-KMS)**
- 



✓ **Block Public Access**

✓ **Lake Formation for table/column access**

✓ **VPC Endpoints (no internet exposure)**

Scenario:

We configured S3 with SSE-KMS + Lake Formation permissions for PII data.

Tips

- Always mention SSE-KMS — it's the expected AWS answer.

9 Question: What challenges did you face in AWS Data Engineering projects?

What they asked me

“What actual challenges have you faced?”

What I said

I mentioned **5 real challenges**:

1. **Small files issue** in S3 → solved using compaction job
2. **Schema drift** → handled using Glue Schema Registry
3. **Long-running Glue jobs** → optimized Spark partitioning
4. **DMS CDC lags** → auto-scaling + batching
5. **Redshift query slowdown** → sorted keys + distribution keys tuning

Tips

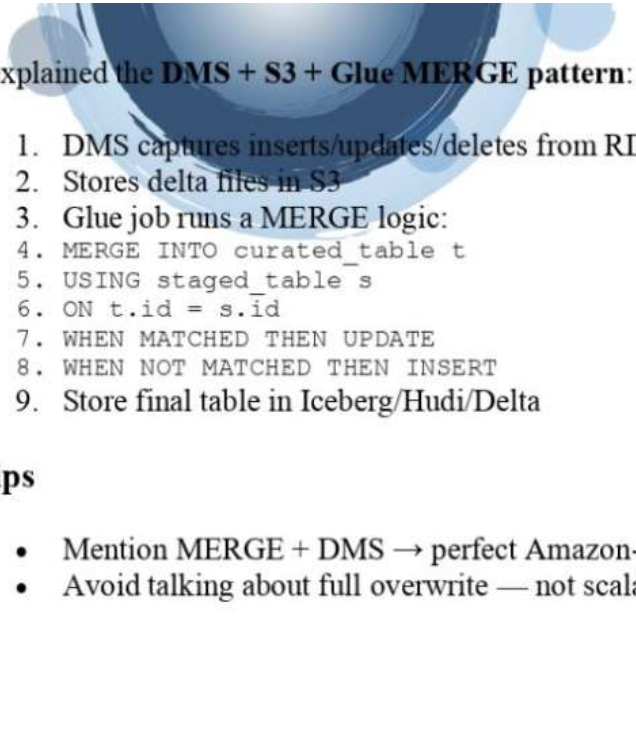
- Real challenges → interviewers love this.
- Avoid saying: “No issues, everything worked fine.”

10 Question: How do you handle CDC (Change Data Capture) in AWS?

What they asked me

“Explain how you process inserts/updates/deletes from a source DB.”

What I said



I explained the **DMS + S3 + Glue MERGE** pattern:

1. DMS captures inserts/updates/deletes from RDS
2. Stores delta files in S3
3. Glue job runs a MERGE logic:
4. `MERGE INTO curated_table t`
5. `USING staged_table s`
6. `ON t.id = s.id`
7. `WHEN MATCHED THEN UPDATE`
8. `WHEN NOT MATCHED THEN INSERT`
9. Store final table in Iceberg/Hudi/Delta

Tips

- Mention MERGE + DMS → perfect Amazon-style architecture.
 - Avoid talking about full overwrite — not scalable.
- 